BreachWatch

Checks whether public companies report material cyberattacks to the SEC within four business days, with every answer traced back to the filing text.

44 Item 1.05 filings

20 companies

Dec 2023–now full rule period

System architecture

Data flows left to right, from the SEC's public database to an AI agent. Solid stages run today; dashed stages are designed and next to build.

Built and working Planned

Source
SEC EDGAR
Free public filing database.
- Full-text search
- Submissions API
- Filing archive

→

Python
Ingestion
`discover_tickers.py` finds filers; `edgar_ingest.py` keeps only Item 1.05 8-Ks and amendments.

→

Snowflake · RAW
Stage and load
Files land in `EDGAR_STAGE`, then `COPY INTO` two RAW tables.

→

dbt · STAGING / MARTS
Clean and model
Strip HTML, extract Item 1.05 text, link amendments into incidents, chunk text.

→

Embeddings + LLM
RAG layer
Vector search over chunks; LLM extracts the materiality date with a quoted source.

→

MCP server
Agent tools
- `get_cyber_incidents`
- `check_disclosure_timeliness`
- `search_filings`

→

Client
AI assistant
Answers "Did this company report on time?" in plain language.

Airflow (planned) will schedule ingestion, loading and dbt runs so new filings flow in automatically.

The hard problem we're designing around

The four-day deadline starts when a company decides an attack is material, not when it happens, and many filings never state that date. Version 1 uses the structured report date from EDGAR; version 2 extracts the materiality date from the filing text and reports "can't tell" when it isn't there.

BreachWatch

Checks whether public companies report material cyberattacks to the SEC within four business days, with every answer traced back to the filing text.

44 Item 1.05 filings

20 companies

Dec 2023–now full rule period

Data model

RAW keeps EDGAR data exactly as received. The modeled layer turns filings into incidents, so an original 8-K and its amendments count as one event.

RAW layer (built, loaded with 44 filings)

FILINGS_METADATA	
record	VARIANT: full JSON from ingestion
source_file	stage file it came from
loaded_at	load timestamp

FILING_DOCUMENTS	
accession_number	SEC filing ID
raw_html	full filing document
source_file	stage file it came from
loaded_at	load timestamp

Modeled layer (planned, built with dbt)

COMPANIES	
cik PK	SEC company ID
ticker	
company_name	

INCIDENTS	
incident_id PK	
cik FK	→ COMPANIES
report_date	shared by original and amendments
materiality_date	LLM-extracted, may be null
materiality_evidence	quote from filing
business_days_to_file	
status	on time / late / can’t tell

FILINGS	
accession_number PK	
incident_id FK	→ INCIDENTS
form_type	8-K or 8-K/A
filing_date	
acceptance_datetime	
source_url	link to EDGAR

FILING_SECTIONS	
section_id PK	
accession_number FK	→ FILINGS
item	e.g. 1.05
section_text	cleaned text

CHUNKS	
chunk_id PK	
section_id FK	→ FILING_SECTIONS
chunk_text	
embedding	VECTOR for semantic search

- One company has many incidents.
- One incident has many filings: the original 8-K plus any 8-K/A amendments, linked by company and report date.
- One filing has many sections, and one section has many chunks.